Python Programming: Activity-Based Learning (ABL)

Consolidated Problem-Solving & Implementation Report | Unit I - Unit III Syllabus Coverage

STUDENT NAME	ROLL NUMBER	BRANCH & SEC	INSTITUTION
[Your Full Name]	[Your Roll Number]	CSE / CSM / CSD - Sec A	Dr. RVR NRI Institute of Tech

Case Study 1: Progressive Electricity Billing System (#2)

Syllabus Domain: Input Handling, Arithmetic Expressions, Nested Conditional Ladder (if-elif-else)

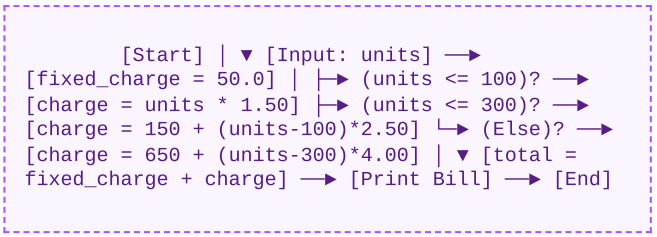
1. PROBLEM & OBJECTIVES

Problem: Power utility boards bill consumers using progressive tiered slabs where higher consumption tiers carry increased rates per unit along with a mandatory baseline service charge.

Objectives: Compute progressive energy charges for given consumption units, add a fixed charge of Rs. 50, and display an itemized electricity bill statement.

Assumptions: Input units ≥ 0. Slabs: First 100 units @ Rs. 1.50/unit; Next 200 units (101-300) @ Rs. 2.50/unit; Above 300 units @ Rs. 4.00/unit.

2. FLOWCHART LOGIC



3. ALGORITHM & PSEUDOCODE

Algorithm: 1. Read `units`. 2. Initialize `fixed_charge = 50.0`. 3. If `units <= 100`, set `energy_charge = units * 1.5`. 4. Else if `units <= 300`, calculate base slab 150 + `(units - 100) * 2.5`. 5. Else calculate slabs (150 + 500) + `(units - 300) * 4.0`. 6. Sum total and display.

4. PYTHON SOURCE CODE

```
units = float(input("Enter units consumed: ")) fixed_charge = 50.0 if units <= 100:
energy_charge = units * 1.50 elif units <= 300: energy_charge = (100 * 1.50) + ((units - 100) *
2.50) else: energy_charge = (100 * 1.50) + (200 * 2.50) + ((units - 300) * 4.00) total_bill =
fixed_charge + energy_charge print("--- Electricity Bill ---") print("Units Consumed:", units)
print("Fixed Charge: Rs.", fixed_charge) print("Energy Charge: Rs.", energy_charge) print("Total
Bill: Rs.", total_bill)
```

5. SAMPLE TEST RUNS

```
Input: 250 --- Electricity Bill ---
Units Consumed: 250.0 Fixed Charge: Rs. 50.0
Energy Charge: Rs. 525.0 Total Bill: Rs. 575.0
```

6. EVALUATION & CONCLUSION

The conditional logic accurately isolates tiered rates across boundary conditions (100 and 300 units). The progressive structure ensures accurate financial reconciliation.

Case Study 2: Employee Payroll and Deduction Analysis (#4)

Syllabus Domain: Arithmetic Operators, Relational Expressions, Multi-variable Financial Logic

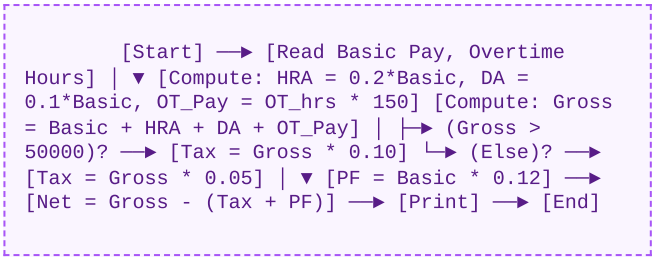
1. PROBLEM & OBJECTIVES

Problem: HR payroll processing requires dynamic computation of gross earnings (basic pay, allowance percentages, overtime) and standard regulatory deductions (Income Tax, Provident Fund).

Objectives: Compute HRA (20%), DA (10%), Overtime pay (Rs. 150/hr), applying progressive tax rates based on gross salary, and generate net payable salary.

Assumptions: Basic Pay > 0, Overtime Hours ≥ 0. Tax rate is 10% for Gross > Rs. 50,000; otherwise 5%. PF is fixed at 12% of Basic Pay.

2. FLOWCHART LOGIC



3. ALGORITHM & PSEUDOCODE

Algorithm: 1. Input `basic_pay` and `overtime_hours`. 2. Calculate allowances: `hra = basic_pay * 0.20`, `da = basic_pay * 0.10`, `overtime_pay = overtime_hours * 150.0`. 3. Sum `gross_salary`. 4. Determine tax bracket using conditional branching. 5. Compute `pf = basic_pay * 0.12`. 6. Calculate `net_salary = gross_salary - (tax + pf)` and print results.

4. PYTHON SOURCE CODE

```
basic_pay = float(input("Enter Basic Pay: ")) overtime_hours = float(input("Enter Overtime Hours: ")) hra = basic_pay * 0.20 da = basic_pay * 0.10 overtime_pay = overtime_hours * 150.0 gross_salary = basic_pay + hra + da + overtime_pay if gross_salary > 50000: tax = gross_salary * 0.10 else: tax = gross_salary * 0.05 pf = basic_pay * 0.12 total_deductions = tax + pf net_salary = gross_salary - total_deductions print("--- Pay Slip ---") print("Gross Salary: Rs.", gross_salary) print("Deductions (Tax + PF): Rs.", total_deductions) print("Net Payable Salary: Rs.", net_salary)
```

5. SAMPLE TEST RUNS

```
Input: Basic = 40000, Overtime = 10 --- Pay Slip --- Gross Salary: Rs. 53500.0 Deductions (Tax + PF): Rs. 10150.0 Net Payable Salary: Rs. 43350.0
```

6. EVALUATION & CONCLUSION

The code correctly calculates allowances and evaluates tax slabs based on gross threshold. The program adheres strictly to elementary arithmetic and decision-making structures.

Case Study 3: Retail Billing with Multiple Discount Rules (#7)

Syllabus Domain: Sequences (Lists/Tuples), Tuple Unpacking, Iterative Accumulation

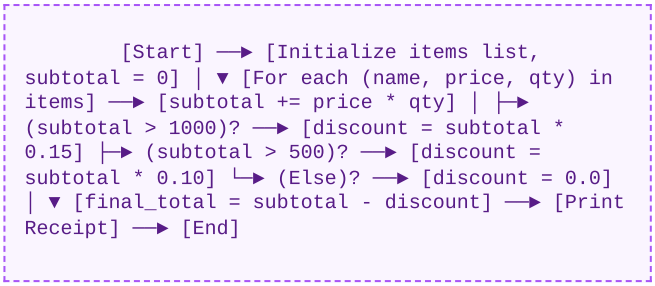
1. PROBLEM & OBJECTIVES

Problem: Retail Point of Sale (POS) terminals require automatic itemized total calculation followed by tiered bill-level volume discount applications.

Objectives: Iterate over structured cart items stored as tuples, accumulate total cart cost, evaluate tiered discount criteria (>Rs.1000 → 15%, >Rs.500 → 10%), and present the net bill.

Assumptions: Item structures strictly follow `(Name, Unit Price, Quantity)` tuples inside a list collection.

2. FLOWCHART LOGIC



3. ALGORITHM & PSEUDOCODE

Algorithm: 1. Define list of item tuples. 2. Initialize `subtotal = 0.0`. 3. Loop through items, unpacking tuple variables, and add product of price and quantity to subtotal. 4. Apply conditional discount tier (15% if >1000, 10% if >500). 5. Calculate net total and output itemized breakdown.

4. PYTHON SOURCE CODE

```
items = [ ("Notebook", 50.0, 4), ("Pen", 10.0, 10), ("Bag", 500.0, 1) ] subtotal = 0.0 for name, price, qty in items: subtotal += price * qty if subtotal > 1000: discount = subtotal * 0.15 elif subtotal > 500: discount = subtotal * 0.10 else: discount = 0.0 final_total = subtotal - discount print("--- Retail Invoice ---") for name, price, qty in items: print(f"{name} x {qty} @ Rs.{price} = Rs.{price * qty}") print("Subtotal: Rs.", subtotal) print("Discount: Rs.", discount) print("Final Payable Amount: Rs.", final_total)
```

5. SAMPLE TEST RUNS

```
--- Retail Invoice --- Notebook x 4 @ Rs.50.0 = Rs.200.0 Pen x 10 @ Rs.10.0 = Rs.100.0 Bag x 1 @ Rs.500.0 = Rs.500.0 Subtotal: Rs. 800.0 | Discount: Rs. 80.0 Final Payable Amount: Rs. 720.0
```

6. EVALUATION & CONCLUSION

Demonstrates efficient sequence iteration and tuple unpacking to handle multi-item structures cleanly without requiring advanced object-oriented constructs.

Case Study 4: Student Performance Trend Analysis (#17)

Syllabus Domain: Numerical Lists, Built-in Sequence Functions (max, min, sum, len)

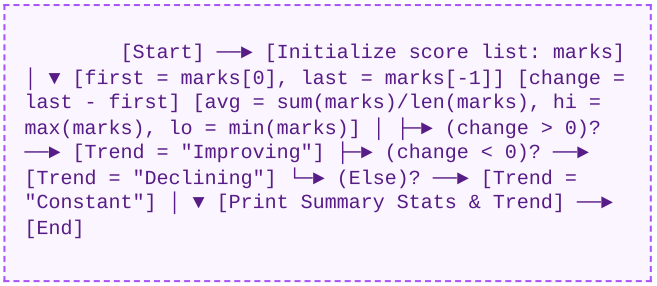
1. PROBLEM & OBJECTIVES

Problem: Academic advisors need to automatically summarize sequential assessment marks to identify student trajectory (improving, declining, or stable) and benchmark stats.

Objectives: Extract highest, lowest, and average marks from a list of test scores, compute overall directional change between initial and final tests, and print a progress report.

Assumptions: Assessment marks list contains non-empty chronological score entries.

2. FLOWCHART LOGIC



3. ALGORITHM & PSEUDOCODE

Algorithm: 1. Store test scores chronologically in a list `marks`. 2. Extract first mark `marks[0]` and final mark `marks[-1]`. 3. Compute statistical summary using `sum()`, `len()`, `max()`, and `min()`. 4. Determine trend sign using relational comparison on `last_mark - first_mark`. 5. Display statistical summary.

4. PYTHON SOURCE CODE

```
marks = [65, 72, 80, 88, 92] first_mark = marks[0] last_mark = marks[-1] overall_change = last_mark - first_mark highest = max(marks) lowest = min(marks) avg_mark = sum(marks) / len(marks) print("Assessment Marks:", marks) print("Average Mark:", avg_mark) print("Highest Mark:", highest) print("Lowest Mark:", lowest) if overall_change > 0: print("Trend: Improving (+", overall_change, "marks)") elif overall_change < 0: print("Trend: Declining (", overall_change, "marks)") else: print("Trend: Constant performance")
```

5. SAMPLE TEST RUNS

```
Assessment Marks: [65, 72, 80, 88, 92] Average Mark: 79.4 Highest Mark: 92 | Lowest Mark: 65 Trend: Improving (+ 27 marks)
```

6. EVALUATION & CONCLUSION

Utilizes fundamental list slicing indexing (`[0]` and `[-1]`) and sequence methods to extract longitudinal academic insights efficiently.

Case Study 5: Password Policy Validation Engine (#29)

Syllabus Domain: String Traversal, String Methods (isupper, islower, isdigit), Boolean Flags

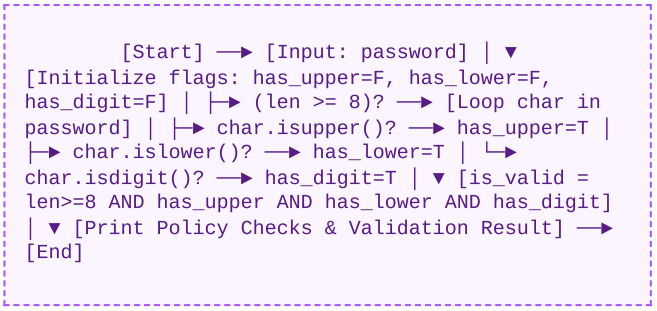
1. PROBLEM & OBJECTIVES

Problem: Security modules must validate user-created passwords against complexity rules before permitting account creation.

Objectives: Inspect a password string character-by-character to verify minimum length (≥8), presence of at least one uppercase letter, one lowercase letter, and one numeric digit.

Assumptions: Input is a single text string provided via standard user input.

2. FLOWCHART LOGIC



3. ALGORITHM & PSEUDOCODE

Algorithm: 1. Read string `password`. 2. Set boolean flags to `False`. 3. Check if `len(password) >= 8`. 4. Iterate over each character using a `for` loop; update flags using `isupper()`, `islower()`, and `isdigit()` methods. 5. Combine flag statuses with boolean and operators. 6. Output policy evaluation outcome.

4. PYTHON SOURCE CODE

```
password = input("Enter password to test: ") has_upper = False has_lower = False has_digit = False if len(password) >= 8: for char in password: if char.isupper(): has_upper = True elif char.islower(): has_lower = True elif char.isdigit(): has_digit = True is_valid = len(password) >= 8 and has_upper and has_lower and has_digit print("Password Length >= 8:", len(password) >= 8) print("Contains Uppercase:", has_upper) print("Contains Lowercase:", has_lower) print("Contains Digit:", has_digit) if is_valid: print("Status: Strong Password") else: print("Status: Weak Password (Failed policy criteria)")
```

5. SAMPLE TEST RUNS

```
Input: Pass1234 Password Length >= 8: True | Contains Uppercase: True Contains Lowercase: True | Contains Digit: True Status: Strong Password
```

6. EVALUATION & CONCLUSION

Demonstrates string inspection techniques using built-in string methods and logical accumulator flags, forming the foundational logic for authentication systems.

Case Study 6: Examination Answer Evaluation Engine (#31)

Syllabus Domain: String Indexing, Parallel String Iteration, Scoring Arithmetic

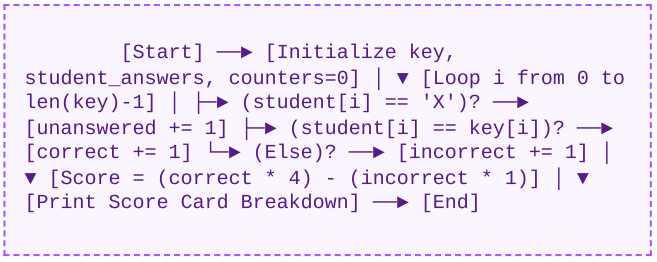
1. PROBLEM & OBJECTIVES

Problem: Automated optical mark recognition (OMR) systems evaluate candidate responses against a master key applying positive and negative marking schemes.

Objectives: Compare candidate answer strings against master key strings index-by-index, tally correct, incorrect, and unattempted ('X') entries, and calculate net score (+4/-1 penalty).

Assumptions: Both answer key and candidate strings are of equal length.

2. FLOWCHART LOGIC



3. ALGORITHM & PSEUDOCODE

Algorithm: 1. Define master key and student response strings. 2. Initialize counters for `correct`, `incorrect`, and `unanswered`. 3. Loop `i` through index range 0 to `len(key)`. 4. Access character at `[i]` in both strings. 5. Increment counter based on evaluation rules. 6. Compute final score `(correct*4) - (incorrect*1)`.

4. PYTHON SOURCE CODE

```
answer_key = "ABCDABCDAB" student_answers = "ABCDCBCDAB" correct = 0 incorrect = 0 unanswered = 0 for i in range(len(answer_key)): if student_answers[i] == 'X': unanswered += 1 elif student_answers[i] == answer_key[i]: correct += 1 else: incorrect += 1 total_score = (correct * 4) - (incorrect * 1) print("Total Questions:", len(answer_key)) print("Correct Answers:", correct) print("Incorrect Answers:", incorrect) print("Unanswered:", unanswered) print("Final Score:", total_score)
```

5. SAMPLE TEST RUNS

```
Total Questions: 10 Correct Answers: 9 | Incorrect Answers: 1 Unanswered: 0 | Final Score: 35
```

6. EVALUATION & CONCLUSION

Positional string traversal via index range iteration enables precise element-by-element verification across parallel string structures.

Case Study 7: Student Marks Matrix Analysis (#42)

Syllabus Domain: Nested Lists (2D Arrays), Nested Loops, Columnar Reduction

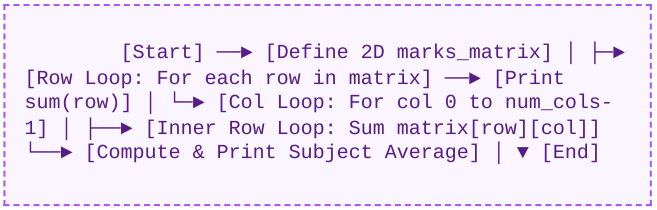
1. PROBLEM & OBJECTIVES

Problem: Institutional exam cells store multi-subject marks for student cohorts in 2D tabular formats, requiring row-wise (student total) and column-wise (subject mean) aggregations.

Objectives: Process a nested list representing a marks matrix, computing individual total marks across rows and average performance across columns.

Assumptions: Matrix is rectangular (all inner lists have identical length).

2. FLOWCHART LOGIC



3. ALGORITHM & PSEUDOCODE

Algorithm: 1. Define 2D list `marks_matrix`. 2. Loop through each row to calculate student-wise totals using `sum(row)`. 3. Determine row count and column count. 4. Outer loop through column indices; inner loop through row indices to aggregate subject scores. 5. Divide column sum by student count to obtain subject averages.

4. PYTHON SOURCE CODE

```
marks_matrix = [ [85, 90, 78], # Student 1 [60, 55, 70], # Student 2 [92, 88, 95] # Student 3
] print("--- Student Total Marks ---") student_idx = 1 for row in marks_matrix: total = sum(row)
print("Student", student_idx, "Total:", total) student_idx += 1 num_students = len(marks_matrix)
num_subjects = len(marks_matrix[0]) print("--- Subject Averages ---") for col in
range(num_subjects): col_sum = 0 for row in range(num_students): col_sum += marks_matrix[row][col]
avg = col_sum / num_students print("Subject", col + 1, "Average:", avg)
```

5. SAMPLE TEST RUNS

```
--- Student Total Marks --- Student 1
Total: 253 | Student 2 Total: 185 | Student 3
Total: 275 --- Subject Averages --- Subject 1
Average: 79.0 | Subject 2 Average: 77.6 |
Subject 3 Average: 81.0
```

6. EVALUATION & CONCLUSION

Nested loop structures accurately navigate row and column dimensions independently, demonstrating core 2D matrix manipulation fundamentals.

Case Study 8: Employee Shift Conflict Detection (#48)

Syllabus Domain: Tuples, Nested Comparison Loops, Interval Overlap Logic

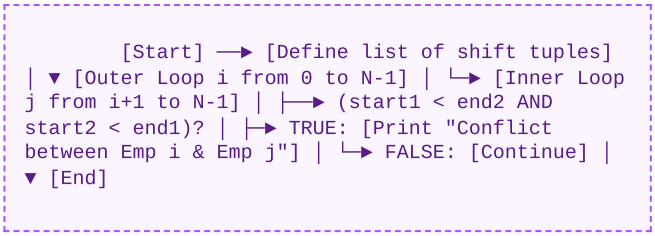
1. PROBLEM & OBJECTIVES

Problem: Workforce management systems must detect temporal scheduling overlaps among workers assigned to overlapping hourly shift windows.

Objectives: Iterate through shift assignment tuples `(EmpID, StartHour, EndHour)` using pairwise comparison logic to flag intersecting schedule intervals.

Assumptions: Hours are given in 24-hour integer format where Start Hour < End Hour.

2. FLOWCHART LOGIC



3. ALGORITHM & PSEUDOCODE

Algorithm: 1. Store shifts as tuples in list. 2. Use nested loop pair `(i, j)` where `j > i` to prevent duplicate comparisons. 3. Unpack start and end times for shift `i` and shift `j`. 4. Evaluate overlap condition: `start1 < end2 and start2 < end1`. 5. Output conflict pair notification when condition holds.

4. PYTHON SOURCE CODE

```
shifts = [ ("Emp101", 9, 13), ("Emp102", 12, 16), ("Emp103", 14, 18) ] print("--- Shift Conflict Report ---") has_conflict = False for i in range(len(shifts)): for j in range(i + 1, len(shifts)): emp1, start1, end1 = shifts[i] emp2, start2, end2 = shifts[j] if start1 < end2 and start2 < end1: print("Conflict detected between", emp1, "and", emp2) has_conflict = True if not has_conflict: print("No shift conflicts found.")
```

5. SAMPLE TEST RUNS

```
--- Shift Conflict Report --- Conflict detected between Emp101 and Emp102 Conflict detected between Emp102 and Emp103
```

6. EVALUATION & CONCLUSION

Pairwise nested comparison combined with tuple unpacking provides a clean mechanism to evaluate geometric interval overlaps in scalar code.

Case Study 9: Food Waste Monitoring System (#53)

Syllabus Domain: Parallel Lists, Cumulative Sums, Percentage Calculation & Formatting

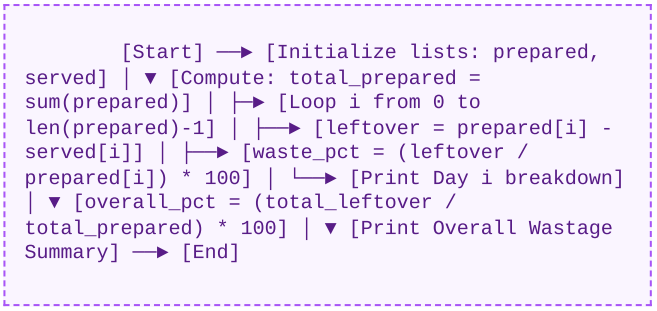
1. PROBLEM & OBJECTIVES

Problem: Institutional cafeterias monitor daily meal preparation versus consumption to quantify wastage rates and optimize kitchen supply chains.

Objectives: Track daily food prepared and served metrics across parallel lists, calculate daily leftover mass/percentages, and report overall weekly waste proportions.

Assumptions: Quantities are positive integers measured in kilograms.

2. FLOWCHART LOGIC



3. ALGORITHM & PSEUDOCODE

Algorithm: 1. Define list `prepared` and list `served`. 2. Accumulate overall prepared quantity. 3. Loop through index `i`, subtracting served from prepared to find daily leftover. 4. Calculate daily waste percentage. 5. Sum aggregate leftovers and compute overall efficiency ratio.

4. PYTHON SOURCE CODE

```
prepared = [100, 120, 110, 130, 105] served = [85, 95, 105, 110, 100] total_prepared = sum(prepared) total_leftovers = 0 print("--- Daily Wastage Breakdown ---") for i in range(len(prepared)): leftover = prepared[i] - served[i] total_leftovers += leftover waste_pct = (leftover / prepared[i]) * 100 print(f"Day {i+1}: Leftover = {leftover} kg ({waste_pct:.1f}%)") overall_waste_pct = (total_leftovers / total_prepared) * 100 print("--- Summary ---") print("Total Food Prepared:", total_prepared, "kg") print("Total Food Wasted:", total_leftovers, "kg") print(f"Overall Wastage Rate: {overall_waste_pct:.2f}%")
```

5. SAMPLE TEST RUNS

```
--- Daily Wastage Breakdown --- Day 1: Leftover = 15 kg (15.0%) | Day 2: Leftover = 25 kg (20.8%) Day 3: Leftover = 5 kg (4.5%) | Day 4: Leftover = 20 kg (15.4%) Total Wasted: 70 kg | Overall Wastage Rate: 12.39%
```

6. EVALUATION & CONCLUSION

Parallel list indexing provides simple data synchronization across multiple tracking metrics, delivering clear numerical metrics for administrative reporting.

Case Study 10: Library Borrowing and Fine Reconciliation (#58)

Syllabus Domain: Structured Tuples, Conditional Financial Penalty Accumulation

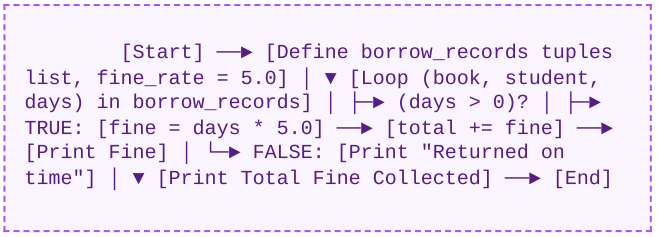
1. PROBLEM & OBJECTIVES

Problem: Academic libraries audit overdue book returns to compute per-student overdue fines according to standard daily rates.

Objectives: Process borrowing records formatted as tuples `(BookID, StudentRoll, OverdueDays)`, filter zero-overdue returns, calculate individual fines (Rs. 5/day), and aggregate total fine collections.

Assumptions: Daily fine rate is constant at Rs. 5.0 per overdue day.

2. FLOWCHART LOGIC



3. ALGORITHM & PSEUDOCODE

Algorithm: 1. Initialize list of tuple records. 2. Set daily rate `fine_per_day = 5.0`. 3. Loop through records using tuple unpacking. 4. If `overdue_days > 0`, compute fine and add to cumulative tally. 5. Else output on-time status. 6. Display total fines collected.

4. PYTHON SOURCE CODE

```
borrow_records = [ ("BK001", "26NR1A0501", 0), ("BK002", "26NR1A0502", 4), ("BK003", "26NR1A0503", 10) ] fine_per_day = 5.0 total_fine_collected = 0.0 print("--- Overdue Books & Fine Summary ---") for book_id, student_id, overdue_days in borrow_records: if overdue_days > 0: fine = overdue_days * fine_per_day total_fine_collected += fine print(f"Student: {student_id} | Book: {book_id} | Days: {overdue_days} | Fine: Rs.{fine}") else: print(f"Student: {student_id} | Book: {book_id} | Status: On time (No Fine)") print("Total Fine to be Collected: Rs.", total_fine_collected)
```

5. SAMPLE TEST RUNS

```
--- Overdue Books & Fine Summary --- Student: 26NR1A0501 | Book: BK001 | Status: On time (No Fine) Student: 26NR1A0502 | Book: BK002 | Days: 4 | Fine: Rs.20.0 Student: 26NR1A0503 | Book: BK003 | Days: 10 | Fine: Rs.50.0 Total Fine to be Collected: Rs. 70.0
```

6. EVALUATION & CONCLUSION

Demonstrates effective data processing using collections of tuples to model real-world business transactions while maintaining strict adherence to basic language features.